OzAgriMate Architecture and Farm API

Pulkit Sharma | ISYS3459 Assignment 4 | 23 October 2026

These sections document the proposed architecture decisions, container and deployment views, and implemented farm API contributed by Pulkit Sharma. The farm endpoints have passed the supplied Postman run. Architecture decisions remain Proposed pending team review.

Design basis and scope

The submitted Assignment 2 report identifies Scalability, Availability and Interoperability as the top three driving characteristics, alongside Security, Elasticity, Reliability / Safety and Performance. Its constraints require Australian processing and residency, offline field operation, managed operations for at most eight engineers and releases without planned seasonal downtime.

Core scope covers telemetry, farm monitoring, irrigation management, alerts and AI-assisted advice. Carbon verification, autonomous dosing, commodity trading and agri-finance remain bounded extension capabilities. They are outside the eight-endpoint prototype; this contribution neither certifies carbon credits nor executes trades, lending decisions or physical dosing.

Farm Owner and Farm Worker have separate responsibilities. Owners control sharing and consent; workers perform explicitly authorised operations. Agronomists and operations managers receive scoped access. Platform administration does not automatically grant permission to read farm records.

Contribution coverage

Artefact	Scope and status
ADR-01	Architectural style; Proposed
ADR-05	Security and farm-scoped authorisation; Proposed
Container and deployment views	Target architecture; supplied as high-resolution figures and SVG sources
Farm API	POST /farms and GET /farms/{farmId}; local implementation
Testing evidence	11 Postman scenarios, 33 assertions passed, zero failures and errors

Production QA targets require separate verification. The local prototype uses in-memory data and fixed demonstration identities; production OAuth, cloud infrastructure and the other six team endpoints are outside this contribution.

ADR 01 Architectural style

Field	Architectural decision record
Title	ADR-01 Adopt service-based architecture with asynchronous telemetry processing
Status	Proposed
Context	OzAgriMate must grow from 5,000 to over 50,000 farms while supporting heterogeneous devices, rural outages, and seasonal load spikes. Constraints C4 and C5 require managed operations by at most eight engineers and releases without planned seasonal downtime. A layered monolith couples scaling and releases; fine-grained microservices increase operational overhead. Coarse-grained services offer a middle ground.
Decision	Adopt service-based architecture with Farm Management, Telemetry, and Irrigation and Alerts services, plus an isolated AI Recommendation service. Use REST for immediate operations and managed durable messaging for telemetry-driven processing. Deploy on managed Australian-region infrastructure. Edge buffering and local approved schedules support disconnected farms. Coarse boundaries permit independent scaling and releases while limiting service proliferation. Isolating AI supports ASR21: baseline monitoring and alerts continue if recommendations fail.
Consequences	Independent scaling and failure isolation support seasonal growth and continuity. Distribution introduces network failures, eventual consistency, duplicate-message handling, contract management, and monitoring costs. Managed infrastructure reduces operational effort but creates vendor dependencies. Broker delivery requires idempotent consumers. The in-memory endpoint prototype demonstrates service structure, not production durability, elasticity, or uptime.
Compliance	Review code boundaries against diagrams; verify API contracts and AI-independent alert flow. Validate seasonal scaling, failover, and rolling releases through production-like load and resilience tests.

Traceability: ASR3, ASR12, ASR15, ASR21, ASR22; constraints C1-C5. This style decision supports several QAs but does not replace the four dedicated QA ADRs.

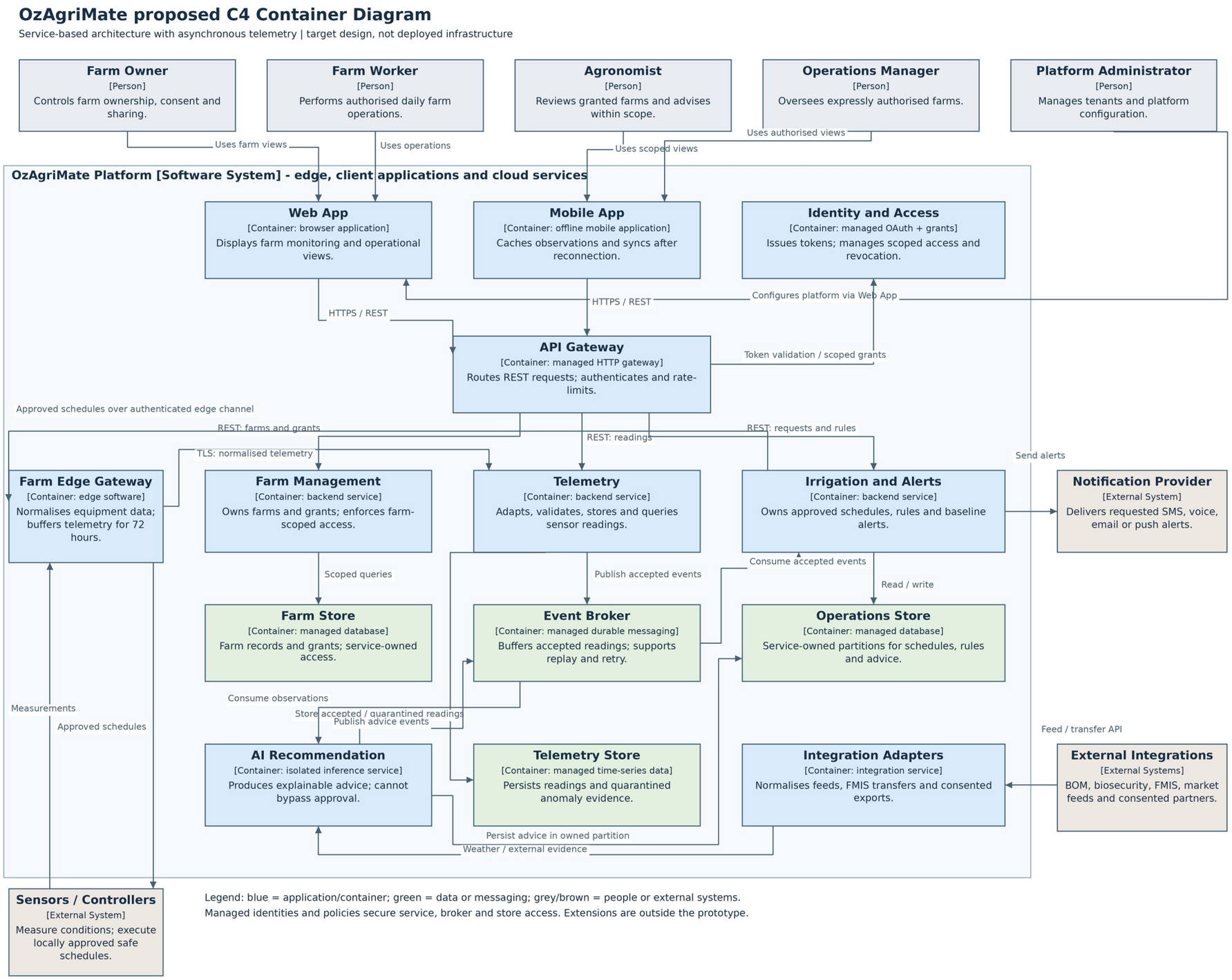
ADR 05 Farm scoped authorisation

Field	Architectural decision record
Title	ADR-05 Enforce farm-scoped authorisation at every service boundary
Status	Proposed
Context	ASR19 requires scoped, time-limited access revocable within one minute; ASR26 requires farm isolation. Owners control access, while workers and agronomists need narrower permissions. Trusting caller-supplied farm identifiers or gateway-only checks risks cross-farm disclosure. Separate databases per farm offer stronger physical separation but increase operational cost at 50,000 farms.
Decision	Use managed OAuth access tokens and service-level authorisation against trusted farm and paddock grants. Restrict repository queries by authorised scope and deny by default. Give grants expiry times and invalidate cached permissions so revocation propagates within one minute. Distinguish owner, worker, agronomist, operations manager, and platform administrator permissions. Platform administration does not automatically grant farm-data access. The prototype substitutes fixed demonstration identities and implements only owner creation and owner-scoped retrieval.
Consequences	Scoped checks reduce cross-farm disclosure and support controlled sharing without per-farm database proliferation. Policy enforcement and grant invalidation add complexity and availability dependencies. Authentication alone does not establish authorisation. Missing and inaccessible farms return identical retrieval errors. Prototype fixtures do not validate OAuth, grant revocation, encryption, audit integrity, or production security.
Compliance	Test permitted and denied farm/paddock access and revocation within one minute. Review repository scoping, validate token controls, and independently assess production isolation.

Traceability: ASR19, ASR26, ASR28; Security driving QA. The referenced ASR identifiers follow the submitted Assignment 2 report.

Container view

Figure 1 OzAgriMate proposed C4 Container Diagram



Container responsibilities and interactions

The container view distributes responsibilities among coarse-grained services and their stores. It shows applications and data stores, consistent with the C4 container abstraction [1]. The three allocated component views should decompose Farm Management, Telemetry, and Irrigation and Alerts.

Container	Responsibility
Farm Management	Owns farm records and grants; applies farm-scoped access.
Telemetry	Adapts and validates sensor readings; stores accepted and quarantined records; publishes accepted events.
Irrigation and Alerts	Owns irrigation requests, approved schedules and alert rules; baseline alerts operate without AI.
AI Recommendation	Consumes observations and external evidence; produces advice and provenance without bypassing approval.
Integration Adapters	Normalises external feeds, FMIS transfer and consented partner exchanges.
Event Broker and stores	Provide durable messaging and service-owned storage in the proposed target environment.

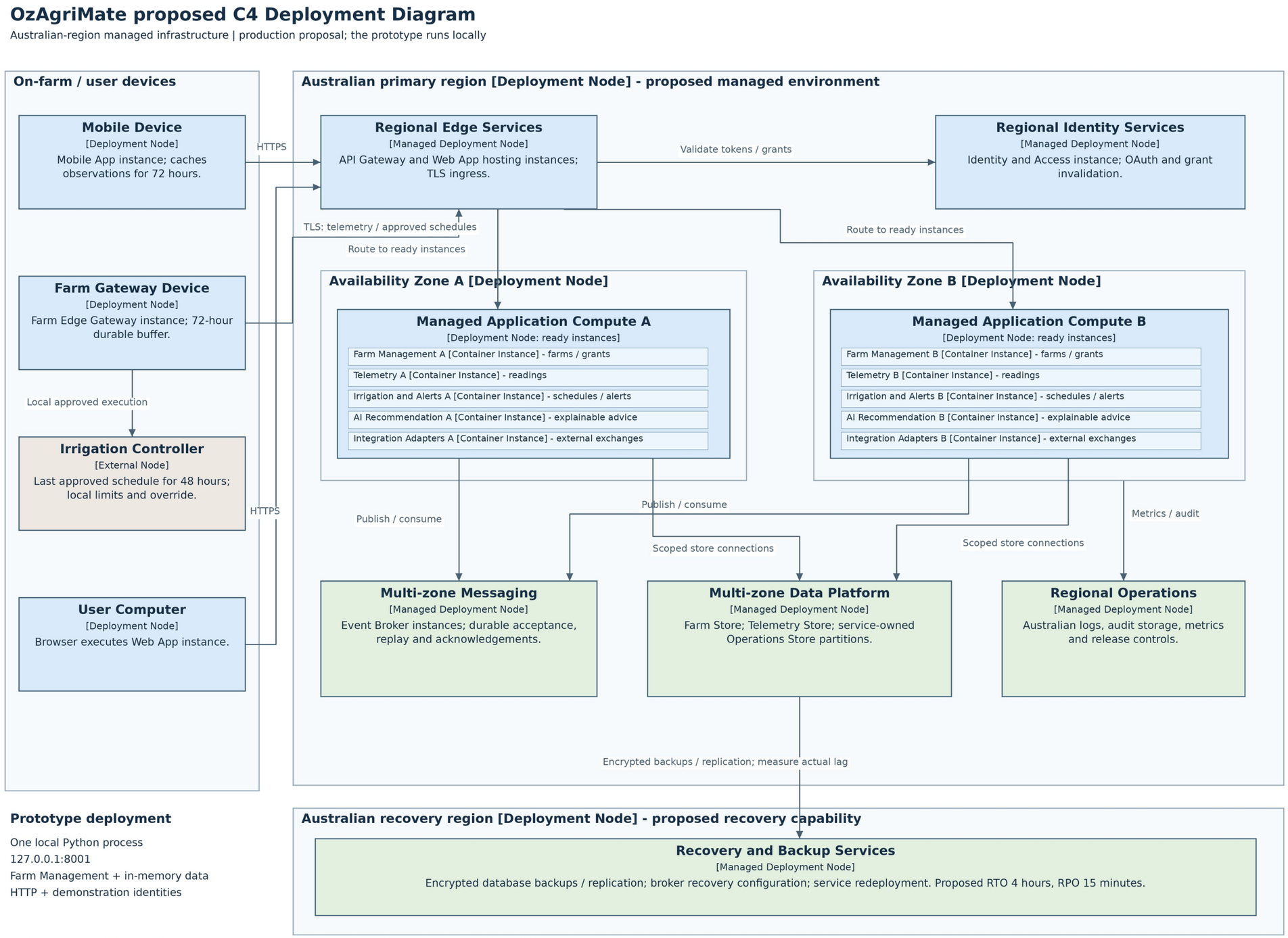
Web and mobile applications call the managed API gateway. The edge gateway normalises field data and buffers it during outages. ISOBUS, ADAPT, LoRaWAN and NB-IoT represent different integration layers and require adapters; they are not interchangeable protocols.

OAuth identity, trusted grants and service-level checks protect synchronous calls. Managed runtime identities and scoped broker/store permissions protect asynchronous processing and data access. The gateway does not replace authorisation inside each service. Service-owned database partitions must use separate credentials and policies.

Accepted-reading and advice events require stable IDs, versioned schemas and idempotent consumers. Suspicious readings require quarantine with reason codes and provenance; they must not silently enter action-triggering flows. Their exact units and thresholds remain a team integration decision.

Deployment view

Figure 2 OzAgriMate proposed C4 Deployment Diagram



Scaling, quorum/failover settings, replica counts, recovery lag and provider residency require validation; no cloud resources have been deployed.

Deployment rationale and implementation boundary

The deployment view maps service instances to a proposed production environment, following C4 deployment conventions [2]. Two availability zones host independently managed backend instances. Managed messaging and data storage support replication and recovery. Rolling releases require compatible API/event schemas and readiness-based traffic routing.

The design places farm data, identity records, telemetry, backups, audit logs and processing in Australian regions to satisfy project constraint C1. Provider capabilities and residency commitments require validation before implementation; this is a project constraint, not a general statement of privacy law.

On-farm gateways retain telemetry for 72 hours; irrigation controllers retain the last approved schedule for 48 hours with local limits and overrides. Buffer capacity and reconnect bandwidth must be sized so replay can meet the proposed fifteen-minute synchronisation target. Physical control and offline operation are not demonstrated by the farm API.

The recovery design targets a four-hour RTO and fifteen-minute RPO. Backup and replication lag, successful restores and regional failover need measured tests before these targets can be claimed as achieved.

Local prototype

Farm Management runs as one standard-library Python process at http://127.0.0.1:8001. The HTTP adapter delegates to an application service and then to an in-memory repository. Fixed bearer fixtures represent demonstration identities. Records reset on restart; multiple independent farm processes would not share state.

Module	Architectural responsibility
app.py	HTTP server and composition root
api.py	Farm REST Controller and error mapping
service.py	Farm Application Service and access decisions
repository.py	Farm Repository with scoped lookup
models.py	Domain types and request validation
security.py	Demonstration identity adapter

Thomas owns the Farm Management component view. It should match these module boundaries. This contribution does not claim his artefacts have been reviewed or that the other services are implemented.

Farm API design

Production gateway versioning is proposed as /api/v1 plus the service paths below. The prototype exposes the unprefixed paths. Each endpoint uses JSON; response fields and status codes match the implementation.

POST /farms

Specification	Value
HTTP Method	POST
Description	Create a farm owned by the authenticated owner
Endpoint URL	/farms
Sample Request Body	{"name":"Riverbend Farm","latitude":-36.36,"longitude":145.4,"areaHectares":125.5}
Sample Response Body	{"id":"farm-001","name":"Riverbend Farm","latitude":-36.36,"longitude":145.4,"areaHectares":125.5}
Success status	201 Created

GET /farms/{farmId}

Specification	Value
HTTP Method	GET
Description	Retrieve details of an accessible farm
Endpoint URL	/farms/{farmId}
Sample Request Body	N/A
Sample Response Body	{"id":"farm-001","name":"Riverbend Farm","latitude":-36.36,"longitude":145.4,"areaHectares":125.5}
Success status	200 OK

The farm-001 example assumes the first creation after server startup. POST supplies a Location header containing the generated resource path. Consumers should use the returned ID. Duplicate names are allowed; names are trimmed. GET returns the same resource representation.

Validation and error behaviour

A creation body must contain exactly name, latitude, longitude and areaHectares. Caller-supplied owner fields are rejected; ownership comes from trusted identity. Names must contain 1 to 120 characters after trimming. Coordinates must be finite numbers within latitude -90 to 90 and longitude -180 to 180. Area must be a finite number greater than zero. Boolean, null and numeric-string values are rejected.

The demonstration accepts Authorization: Bearer demo-owner-a. The separate demo-owner-b identity cannot retrieve owner A records. demo-worker-a is denied farm creation. Missing and inaccessible farms deliberately return the same retrieval error. Paddock grants, agronomist sharing, token expiry and one-minute revocation are proposed production capabilities rather than completed prototype controls.

Status	Code	Relevant condition
400	INVALID_JSON	Malformed UTF-8 JSON or literal NaN/Infinity
401	UNAUTHENTICATED	Missing or invalid demonstration bearer identity
403	FORBIDDEN	Worker attempts farm creation
404	FARM_NOT_FOUND	Farm absent or inaccessible
404	ROUTE_NOT_FOUND	Unknown path or malformed route identifier
405	METHOD_NOT_ALLOWED	Unsupported method; Allow header identifies supported method
413	BODY_TOO_LARGE	Body exceeds 65,536 bytes
415	UNSUPPORTED_MEDIA_TYPE	Content-Type is not application/json
422	VALIDATION_ERROR	Missing/extra fields or invalid values

Transport handling also rejects invalid lengths, unsupported chunked framing and incomplete bodies; a timed-out body read returns 408. These transport checks supplement the endpoint contract.

Error response example:

{"error":{"code":"FARM_NOT_FOUND","message":"Farm not found or not accessible."}}

Verified Postman execution

Pulkit supplied the original screenshot of a completed Postman web run using Desktop Agent against the local farm API. The taskbar date is 23 October 2026 and the run time displayed is 03:11:47 AM. The screenshot records the following result.

Measure	Observed result
Request scenarios	11 across two endpoint operations
Assertions passed	33
Assertions failed / skipped	0 / 0
Run errors	0
Iterations	1
Displayed duration	2 seconds 166 milliseconds
Displayed average response time	5 milliseconds

The visible detail confirms creation returned 201 and retrieval returned 200, including a check that the retrieved record matches creation. Cross-owner retrieval returned the expected 404. All listed assertions passed. The collection also includes missing authentication, worker denial, negative area, caller ownership injection, malformed JSON and wrong media type scenarios.

The reported latency belongs to one local functional run. It is not evidence of performance under 50,000 farms, seasonal peaks or rural network conditions. Ten earlier HTTP integration tests passed in the assistant environment; their historical verification is separate from this user-run Postman evidence.

The original PNG is retained unchanged as evidence/Postman_Run_2026-10-23.png. Evidence_Record.json records its SHA-256 digest and source identifier, allowing later copies to be checked for accidental changes.

Screenshot SHA-256: 8923891397fac46ea87ac9f5411d193bdcf8a2fd7b094f6fba91e25d07a95e8f

The screenshot is retained separately to preserve its original resolution and avoid reproducing unrelated browser information within the report.

References

[1] Brown, S. (n.d.). Container diagram. C4 model. https://c4model.com/diagrams/container. Accessed 23 October 2026.

[2] Brown, S. (n.d.). Deployment diagram. C4 model. https://c4model.com/diagrams/deployment. Accessed 23 October 2026.

[3] Sharma, P., Singh, A., Wight, T., and Shears, B. (2026). OzAgriMate Software Architecture Proposal. Submitted ISYS3459 Assignment 2 report supplied by Pulkit.

[4] RMIT University. (2026). ISYS3459 Assignment 4 Software Architecture Project activity instructions, pp. 1-4. Supplied course material.

[5] RMIT University. (2026). ADR Template. Supplied course material; template cites Head First Software Architecture (2024) and Fundamentals of Software Architecture (2020).

[6] Pulkit Sharma. (2026). Postman farm API run screenshot, 23 October 2026. Original evidence retained with this contribution.